Horizon Data — Unimplemented API Endpoints

All endpoints are relative to the Flask server base URL (`http://localhost:8080`). All request/response bodies are `application/json` . Authentication endpoints use **HTTP-only session cookies** (`credentials: 'include'` on the client); all other endpoints require the user to be authenticated via cookie.

Auth

1. `POST /api/auth/login`

Purpose: Authenticate a user with email and password. Creates a server-side session and sets a session cookie.

Request body:

```
{
  "email": "teacher@latrobe.edu.au",
  "password": "password123"
}
```

Success response `200 OK` :

```
{
  "success": true,
  "user": {
    "id": "1",
    "email": "teacher@latrobe.edu.au",
    "fullName": "Dr. Jane Smith",
    "role": "teacher",
    "phone": "+61 412 345 678",
    "institution": "La Trobe University",
    "is2FAEnabled": false,
    "notificationsEnabled": true
  }
}
```

Failure response `401 Unauthorized` :

```
{
  "success": false,
  "error": "Invalid email or password"
}
```

Notes: Must set an HTTP-only session cookie. Do not return the password field.

---

2. `POST /api/auth/logout`

Purpose: Invalidate the current session and clear the session cookie.

Request body: None.

Success response `200 OK` :

```
{
  "success": true
}
```

Notes: Should succeed even if no session exists (idempotent).

---

3. `GET /api/auth/session`

Purpose: Check whether the caller has a valid session. Called on every page load to restore auth state.

Request body: None.

Success response `200 OK` (authenticated):

```
{
  "authenticated": true,
  "user": {
    "id": "1",
    "email": "teacher@latrobe.edu.au",
    "fullName": "Dr. Jane Smith",
    "role": "teacher",
    "phone": "+61 412 345 678",
    "institution": "La Trobe University",
    "is2FAEnabled": false,
    "notificationsEnabled": true
  }
}
```

Response when not authenticated `401 Unauthorized` :

```
{
  "authenticated": false
}
```

Account

4. `GET /api/account`

Purpose: Return the profile details for the currently authenticated user.

Request body: None.

Success response `200 OK` :

```
{
  "id": "1",
  "email": "teacher@latrobe.edu.au",
```

```
  "fullName": "Dr. Jane Smith",
  "phone": "+61 412 345 678",
  "institution": "La Trobe University",
  "is2FAEnabled": false,
  "notificationsEnabled": true
}
```

Notes: Derived from the session — returns data for the currently logged-in user only.

---

5. `PUT /api/account`

Purpose: Update the profile details of the currently authenticated user.

Request body:

```
{
  "fullName": "Dr. Jane Smith",
  "email": "jane.smith@latrobe.edu.au",
  "phone": "+61 412 345 678",
  "institution": "La Trobe University",
  "is2FAEnabled": false,
  "notificationsEnabled": true
}
```

Success response `200 OK` — returns the updated account object (same shape as `GET /api/account`).

Notes: All fields are optional; only provided fields need updating.

Bookings

6. `GET /api/bookings`

Purpose: Return all bookings for the currently authenticated teacher, grouped by status.

Request body: None.

Success response `200 OK`:

```
{
  "upcoming": [
    {
      "id": 1,
      "date": "18/04/2026",
      "time": "20:00 - 21:30",
      "status": "Confirmed",
      "statusColor": "confirmed",
      "title": "Year 9 Science Class",
      "description": "Introduction to telescope operation."
    }
  ],
  "past": [
    {
```

```
      "id": 3,
      "date": "08/04/2026",
      "time": "21:00 - 22:30",
      "status": "Completed",
      "statusColor": "completed",
      "title": "Year 10 - Jupiter Observation",
      "description": "Planetary observation session.",
      "captureCount": 12
    }
  ],
  "pending": [
    {
      "id": 2,
      "date": "22/04/2026",
      "time": "19:30 - 21:00",
      "status": "Pending",
      "statusColor": "pending",
      "title": "Evening Star Party",
      "description": "After-school astronomy club."
    }
  ]
}
```

Notes:

- `statusColor` values consumed by the frontend: `"confirmed"`, `"pending"`, `"completed"`.
- `captureCount` is only needed on past bookings.
- Date format must be `DD/MM/YYYY`. Time format must be `HH:MM - HH:MM`.

---

7. `POST /api/bookings`

Purpose: Create a new booking request (initially in `pending` state awaiting approval).

Request body:

```
{
  "date": "2026-04-25",
  "startTime": "20:00",
  "endTime": "21:30",
  "title": "ANZAC Day Star Party",
  "description": "Optional description",
  "targets": [
    { "id": "jupiter", "name": "Jupiter" },
    { "id": "saturn", "name": "Saturn" }
  ],
  "targetCount": 2
}
```

Success response `201 Created`:

```
{
  "success": true,
  "id": 12345,
  "booking": {
    "id": 12345,
    "date": "25/04/2026",
    "time": "20:00 – 21:30",
    "status": "Pending",
    "statusColor": "pending",
    "title": "ANZAC Day Star Party",
    "description": "Optional description",
    "targetCount": 2,
    "targets": [
      { "id": "jupiter", "name": "Jupiter" },
      { "id": "saturn", "name": "Saturn" }
    ]
  }
}
```

Notes: `date` arrives as `YYYY-MM-DD` and should be stored/returned as `DD/MM/YYYY`.

---

8. `GET /api/bookings/availability`

Purpose: Return available 30-minute night-time slots for a given date range, excluding already-booked slots. Used to populate the booking calendar.

Query parameters:

Param	Format	Example
`startDate`	`YYYY-MM-DD`	`2026-04-21`
`endDate`	`YYYY-MM-DD`	`2026-04-27`

Success response `200 OK`:

```
{
  "slots": [
    { "date": "2026-04-21", "startTime": "18:00", "endTime": "18:30" },
    { "date": "2026-04-21", "startTime": "18:30", "endTime": "19:00" }
  ],
  "total": 84,
  "range": { "startDate": "2026-04-21", "endDate": "2026-04-27" }
}
```

Notes:

- Night slots only: `18:00–23:30` (evening) and `00:00–05:30` (early morning).
- Slots that overlap with any existing confirmed or pending booking must be excluded.
- The frontend treats a `404` response as "all slots available" (graceful degradation).

---

9. `GET /api/bookings/:id`

Purpose: Return details for a single booking by ID. Used by the session lobby to display session info.

Path parameter: `id` — integer booking ID.

Success response `200 OK` — same booking object shape as items in `GET /api/bookings`.

Failure response `404 Not Found`:

```
{
  "success": false,
  "error": "Booking not found"
}
```

Sessions

Sessions are runtime state associated with a booking. A session is created (or retrieved) when the teacher opens the lobby. It tracks a join code and the list of participants. Sessions are keyed by `bookingId`.

10. `GET /api/sessions/:id`

Purpose: Get or create the session for a booking. Called by both the teacher lobby (to get the join code) and the student lobby (to poll for session status).

Path parameter: `id` — booking ID.

Success response `200 OK`:

```
{
  "success": true,
  "session": {
    "bookingId": 1,
    "joinCode": "789012",
    "status": "waiting",
    "participantCount": 3,
    "createdAt": "2026-04-20T20:00:00.000Z"
  }
}
```

Notes:

- `status` values: `"waiting"` (lobby open, not started), `"active"` (teacher has started the session), `"ended"`.
- If no session exists for the booking, create one and generate a 6-digit numeric join code.
- The student lobby polls this endpoint every 3 seconds and navigates to `/live/student` when `status === "active"`.

---

11. `GET /api/sessions/:id/participants`

Purpose: Return the list of students who have joined the session. Polled every 3 seconds by the teacher lobby.

Path parameter: `id` — booking ID.

Success response `200 OK`:

```
{
  "success": true,
  "participants": [
    { "id": "anon-1234", "name": "Emma Wilson", "joinedAt": "2026-04-
20T20:02:00.000Z" },
    { "id": "anon-5678", "name": "Liam Chen", "joinedAt": "2026-04-20T20:03:00.000Z"
}
  ],
  "total": 2
}
```

Notes: Returns an empty `participants` array (not a 404) if no session exists yet.

---

12. `POST /api/sessions/lookup`

Purpose: Resolve a 6-digit join code to a booking ID. Called by students on the join page before they enter their name.

Request body:

```
{
  "joinCode": "789012"
}
```

Success response `200 OK`:

```
{
  "success": true,
  "bookingId": 1,
  "status": "waiting"
}
```

Failure response `404 Not Found`:

```
{
  "success": false,
  "error": "Invalid join code"
}
```

Failure response `400 Bad Request` (missing code):

```
{
  "success": false,
  "error": "Join code required"
}
```

Notes: Students are unauthenticated — this endpoint must not require a session cookie.

---

13. `POST /api/sessions/:id/join`

Purpose: Register a student as a participant in a session. Called after the student has entered their name.

Path parameter: `id` — booking ID (returned by `POST /api/sessions/lookup`).

Request body:

```
{
  "joinCode": "789012",
  "name": "Emma Wilson"
}
```

Success response `200 OK`:

```
{
  "success": true,
  "message": "Joined session successfully",
  "session": {
    "bookingId": 1,
    "joinCode": "789012",
    "status": "waiting"
  }
}
```

Failure response `404 Not Found` — session not found.

Failure response `403 Forbidden` — join code does not match:

```
{
  "success": false,
  "error": "Invalid join code"
}
```

Notes: Students are unauthenticated. Each call creates a new participant record — there is no deduplication requirement at this stage.

---

14. `POST /api/sessions/:id/start`

Purpose: Mark a session as active. Called by the teacher when they click "Begin Session" in the lobby. Causes all polling student lobbies to redirect to the live view.

Path parameter: `id` — booking ID.

Request body: None.

Success response `200 OK`:

```
{
  "success": true,
  "session": {
    "bookingId": 1,
    "joinCode": "789012",
    "status": "active",
    "startedAt": "2026-04-20T20:15:00.000Z"
  }
}
```

Failure response `404 Not Found` — session not found.

Notes: Requires teacher authentication. After this call, `GET /api/sessions/:id` must return `status: "active"`.

Implementation Notes

- **Session storage:** The session endpoints (`/api/sessions/*`) require in-memory or database-backed state that persists at least for the duration of a booking. An in-memory store (dict/map keyed by `bookingId`) is sufficient for an MVP.
- **Join codes:** 6-digit random numeric string, e.g. `str(random.randint(100000, 999999))` . Must be unique per active session.
- **Student auth:** The student join flow (`/api/sessions/lookup` , `/api/sessions/:id/join`) is intentionally unauthenticated.
- **Existing Flask patterns:** Follow the blueprint/factory pattern in `server/app/` . Add a `sessions_bp` blueprint at prefix `/api/sessions` and an `auth_bp` at `/api/auth` . The account endpoint fits within an existing or new `users_bp` at `/api` .